

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Работа с файлове

Отваряне и затваряне на файлове

Python поддържа много различни типове файлове. Те се разделят основно на два типа:

- **текстови** - cvs, txt, html и други, т.е. файлове, които съхраняват информация в текстова форма;
- **двоични** - файлове съхраняващи информация под формата на изображения, аудио и видео файлове и други.

Работа с файлове, изисква определена последователност от операции:

1. Отваряне на файл с метода `open()`.
2. Четене на файл с метода `read()` или запис във файл с метода `write()`.
3. Затваряне на файл с метода `close()`.

Функцията `open()` има следната формална дефиниция:

`open(file, mode)`

file - първият аргумент на функцията е пътят към файла.

mode - вторият аргумент на функцията е режима на отваряне на файла, той е различен според операцията, която ще извършваме с файла.

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

file може да бъде:

- **Абсолютен**, т.е. да започва с буквата на устройството:

C://somedir/somefile.txt .

- **Относителен**:

somedir/somefile.txt

По този начин търсенето на файла ще бъде спрямо местоположението на изпълнявания скрипт на Python.

mode има 4 режима за отваряне:

- **r (За четене);**

Файлът се отваря за четене. Ако файлът не бъде намерен, тогава се създава изключение **FileNotFoundError**.

- **w (За запис);**

Файлът се отваря за запис. Ако файлът не съществува, тогава той се създава. Ако такъв файл вече съществува, той се създава наново и съответно старите данни в него се изтриват.

- **a (За добавяне);**

Файлът се отваря за запис. Ако файлът не съществува, тогава той се създава. Ако подобен файл вече съществува, тогава данните се записват до края му.

- **b (За двоични файлове).**

Използва се за работа с двоични файлове. Използва се заедно с други режими - **w** или **r**.

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

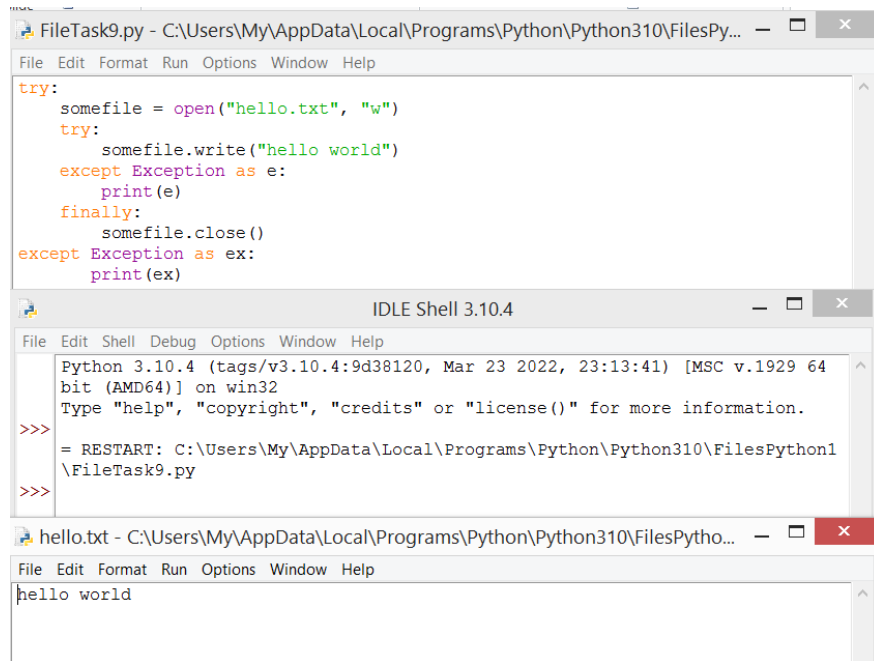
След като приключи работата с файлът, той трябва да бъде затворен с помощта на метод `close()`. Този метод ще освободи всички използвани ресурси, свързани с файла.

Пример: Отворете текстов файл "hello.txt" за писане:

```
myfile = open("hello.txt", "w")  
myfile.close()
```

При отваряне на файл или при работа с него може да се срещнат различни изключения, например да няма достъп до него и други. В този случай програмата генерира грешка и нейното изпълнение не достига извикването на метода за затваряне и съответно файлът няма да бъде затворен. В конкретният случай е възможно да се обработят изключенията:

```
try:  
    somefile = open("hello.txt", "w")  
try:  
    somefile.write("hello world")  
except Exception as e:  
    print(e)  
finally:  
    somefile.close()  
except Exception as ex:  
    print(ex)
```



The screenshot shows a Python IDE with three windows. The top window, titled 'FileTask9.py', contains the following code:

```
try:  
    somefile = open("hello.txt", "w")  
    try:  
        somefile.write("hello world")  
    except Exception as e:  
        print(e)  
    finally:  
        somefile.close()  
except Exception as ex:  
    print(ex)
```

The middle window, titled 'IDLE Shell 3.10.4', shows the output of the script:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64  
bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>> = RESTART: C:\Users\My\AppData\Local\Programs\Python\Python310\FilesPython1  
\FileTask9.py  
>>>
```

The bottom window, titled 'hello.txt', shows the content of the file:

```
hello world
```

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Така работата с файлът преминава във вложен блок, и при внезапно възникване на изключение, файлът се затваря в блока **finally**.

Съществува по-удобна конструкция - **with** конструкция:

with open(file, mode) as file_obj:

инструкции

Тази конструкция дефинира променливата **file_obj** за отворен файл и изпълнява набор от инструкции. След тяхното изпълнение файлът се затваря автоматично. Дори, ако възникнат изключения по време на изпълнение на операторите в блока **with**, файлът все още е затворен. Предходния пример ще добие вида:

with open("hello.txt", "w") as somefile:

somefile.write("hello world")

Текстови файлове

Писане в текстов файл

За да се отвори текстов файл за запис, трябва да се използва **режима w** (**презаписване**) или (**добавяне**). След това методът **write(str)** се използва за запис, на който се предава низът за запис. Трябва да се отбележи, че се записва низът, следователно, ако трябва да се пишат числа, данни от друг тип, тогава те първо трябва да бъдат преобразувани в низ. Нека да се добави информация във файла "hello.txt":

with open("hello.txt", "w") as file:

file.write("hello world")

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Ако се отвори папката, в която се намира текущият скрипт на Python, ще се види файлът **hello.txt**. Този файл може да бъде отворен във всеки текстов редактор и да се промени при желание. Сега нека добавим още един ред към този файл:

```
with open("hello.txt", "a") as file:  
    file.write("\ngood bye, world")
```

Добавянето изглежда, като добавяне на низ към последния знак във файла, така че ако трябва да се пише на нов ред, може да се използва **escape** последователността **"\n"**.

В резултат на това файлът **hello.txt** ще има следното съдържание:

```
hello world  
good bye, world
```

Друг начин за запис във файл е стандартният метод **print()**, който се използва за отпечатване на данни в конзолата:

```
with open("hello.txt", "a") as hello_file:  
    print("Hello, world", file=hello_file)
```

За да се изведат данните във файл, името на файла се предава на метода за печат като втори параметър през параметъра на файла. Първият параметър представлява низа, който трябва да бъде записан във файла.

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

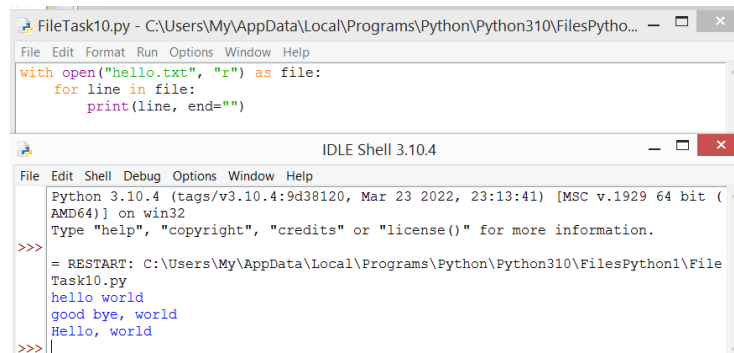
Четене на файл

За да се прочете файл, той се отваря с режим `r` (Четене). Неговото съдържание може да се прочете с помощта на различни методи:

- **`readline()`**: чете един ред от файл
- **`read()`**: чете цялото съдържание на файл на един ред
- **`readlines()`**: чете всички редове на файл в списък

Пример: Ако се налага да се чете записания по-горе файл ред по ред:

```
with open("hello.txt", "r") as file:  
    for line in file:  
        print(line, end="")
```



The screenshot shows a Python IDLE Shell window. The top pane displays the code from the previous block. The bottom pane shows the output of the script, which reads the contents of 'hello.txt' line by line and prints them. The output is: `hello world`, `good bye, world`, and `Hello, world`.

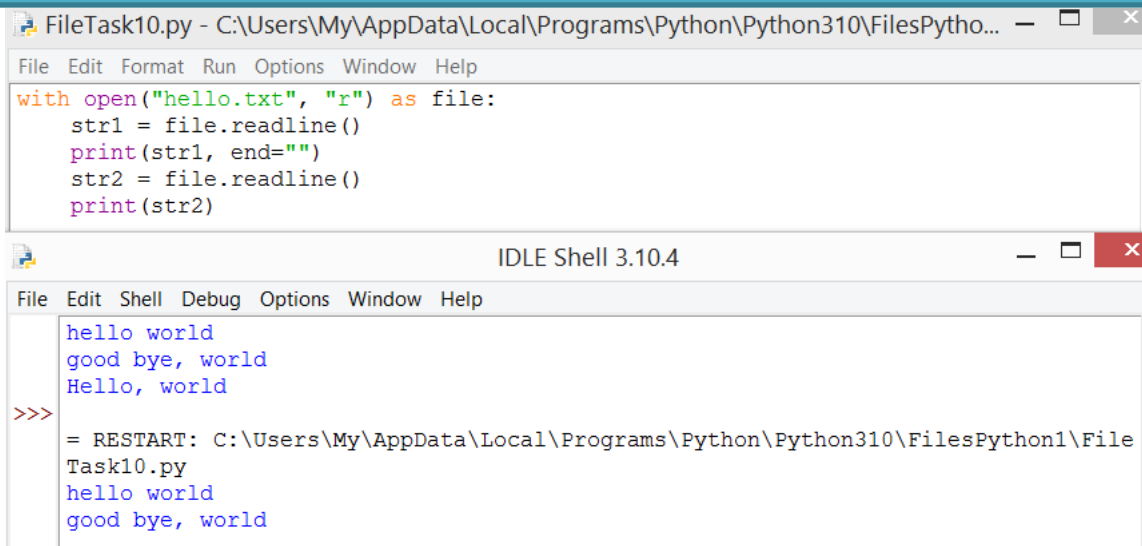
Въпреки, че не се използва изрично метода **`readline()`** за четене на всеки ред, когато се итерира през файла, този метод се извиква автоматично, за да получи всеки нов ред. Следователно няма смисъл ръчно да се извика метода **`readline`** в цикъла. И тъй като редовете са разделени от символа за нов ред `"\n"`, то за да се избегне ненужно прехвърляне към друг ред, стойността се предава на функцията за печат **`end=""`**. Нека изрично да се извика метода **`readline()`** за четене на отделни редове:

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

```
with open("hello.txt", "r") as file:  
    str1 = file.readline()  
    print(str1, end="")  
    str2 = file.readline()  
    print(str2)
```

Изход на конзолата:

```
hello world  
good bye, world
```



The screenshot shows a Python IDE window titled 'FileTask10.py'. The code in the editor is:

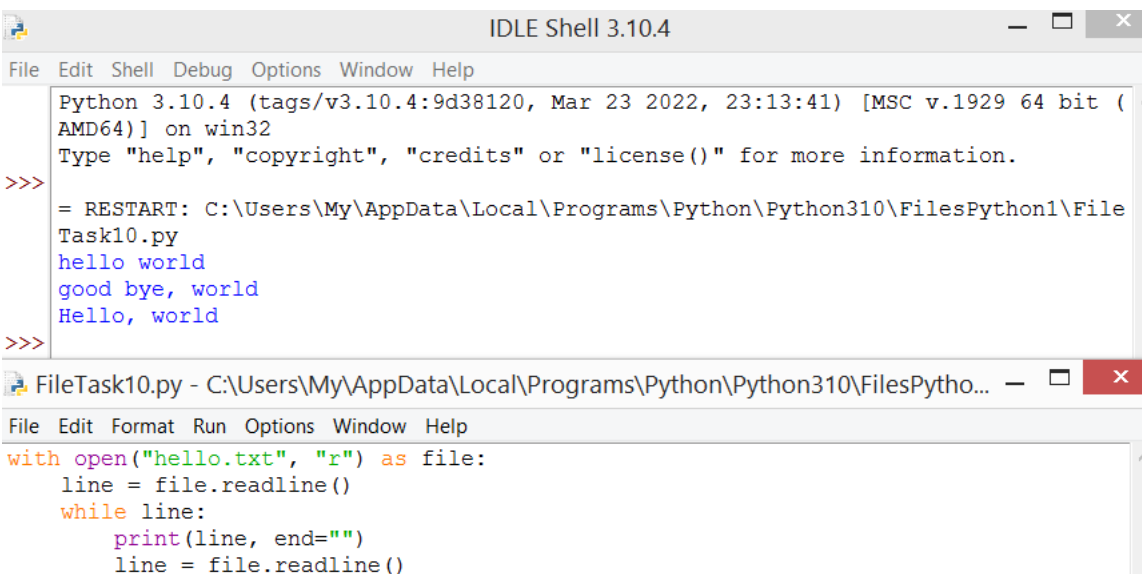
```
with open("hello.txt", "r") as file:  
    str1 = file.readline()  
    print(str1, end="")  
    str2 = file.readline()  
    print(str2)
```

Below the editor is the 'IDLE Shell 3.10.4' window. It shows the output of the script:

```
>>>  
hello world  
good bye, world  
Hello, world  
>>>  
= RESTART: C:\Users\My\AppData\Local\Programs\Python\Python310\FilesPython1\File  
Task10.py  
hello world  
good bye, world
```

Методът `readline` може да се използва за четене на файл ред по ред в цикъл `while`:

```
with open("hello.txt", "r") as file:  
    line = file.readline()  
    while line:  
        print(line, end="")  
        line = file.readline()
```



The screenshot shows a Python IDE window titled 'FileTask10.py'. The code in the editor is:

```
with open("hello.txt", "r") as file:  
    line = file.readline()  
    while line:  
        print(line, end="")  
        line = file.readline()
```

Below the editor is the 'IDLE Shell 3.10.4' window. It shows the output of the script:

```
>>>  
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>  
= RESTART: C:\Users\My\AppData\Local\Programs\Python\Python310\FilesPython1\File  
Task10.py  
hello world  
good bye, world  
Hello, world  
>>>
```

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Ако файлът е малък, тогава той може да бъде прочетен наведнъж с помощта на метода `read()`:

```
with open("hello.txt", "r") as file:  
    content = file.read()  
    print(content)
```

Ако се използва метода **`readlines()`**, ще се прочете целия файл в списък с редове:

```
with open("hello.txt", "r") as file:  
    contents = file.readlines()  
    str1 = contents[0]  
    str2 = contents[1]  
    print(str1, end="")  
    print(str2)
```

При четене на файл може да се сблъскаме с факта, че кодирането му не съвпада с ASCII. В този случай може изрично да се посочи кодирането с помощта на параметъра за кодиране:

```
filename = "hello.txt"  
with open(filename, encoding="utf8") as file:  
    text = file.read()
```


Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Нека да се създаде малък скрипт, който ще запише масива от низове, въведени от потребителя, и ще го прочетем обратно от файла към конзолата:

#FileTask10

име на файла

```
FILENAME = "messages.txt"
```

дефинирайте празен списък

```
messages = list()
```

```
for i in range(4):
```

```
    message = input("Въведете низа " + str(i+1) + ": ")
```

```
    messages.append(message + "\n")
```

запис на списъка във файл

```
with open(FILENAME, "a") as file:
```

```
    for message in messages:
```

```
        file.write(message)
```

четене на съобщения от файл

```
print("Прочетени съобщения")
```

```
with open(FILENAME, "r") as file:
```

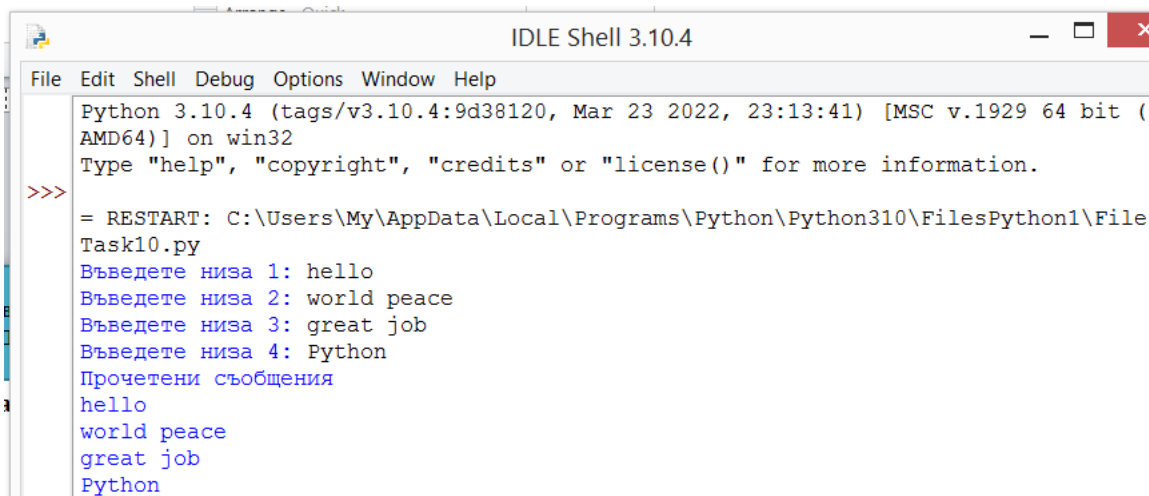
```
    for message in file:
```

```
        print(message, end="")
```

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Пример за работата на програмата:

Въведете низ 1: hello
Въведете низ 2: world peace
Въведете низ 3: great job
Въведете низ 4: Python
Прочетени съобщения
hello
world peace
great job
Python



```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\My\AppData\Local\Programs\Python\Python310\FilesPython1\File Task10.py
Въведете низа 1: hello
Въведете низа 2: world peace
Въведете низа 3: great job
Въведете низа 4: Python
Прочетени съобщения
hello
world peace
great job
Python
```

CSV файлове

Един от често срещаните файлови формати, които съхраняват информация по удобен начин, е **csv** форматът. Всеки ред в **csv** файла представлява един запис или ред, който се състои от отделни колони, разделени със запетаи. Ето защо форматът се нарича стойностен, разделен със запетая. Но въпреки че **csv** форматът е текстов формат, Python предоставя специален вграден **csv** модул, за да улесни работата.

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Примерен модул:

```
import csv
```

```
FILENAME = "users.csv"
```

```
users = [  
    ["Tom", 28],  
    ["Alice", 23],  
    ["Bob", 34]  
]
```

```
with open(FILENAME, "w", newline="") as file:  
    writer = csv.writer(file)  
    writer.writerows(users)
```

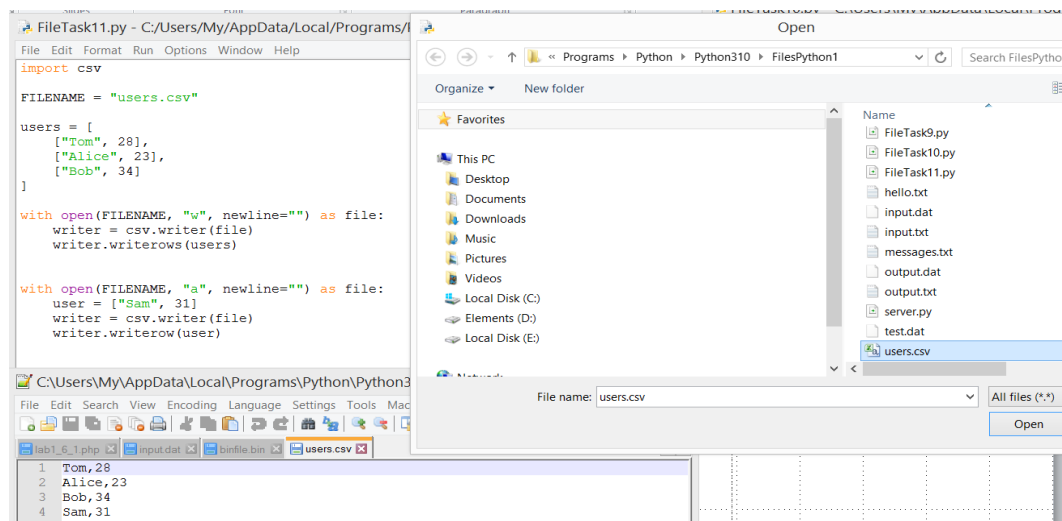
```
with open(FILENAME, "a", newline="") as file:  
    user = ["Sam", 31]  
    writer = csv.writer(file)  
    writer.writerow(user)
```

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Във файла се записва двуизмерен списък - всъщност таблица, където всеки ред представлява един потребител. И всеки потребител съдържа две полета - име и възраст. Това всъщност е таблица от три реда и две колони. При отваряне на файл за запис, стойността **newline=""** се посочва като трети параметър - празен низ, което позволява да се чете правилно редове от файла, независимо от операционната система. За да се пише, трябва да се получи обекта **writer**, който се връща от **csv.writer(file)**. Към тази функция се предава отворен файл. И всъщност записът се прави с помощта на метод **writer.writerows(users)**.

Този метод приема набиране на линии. В нашия случай това е двуизмерен списък. Ако трябва да добавите един запис, който е едномерен списък, например, ["Sam", 31], в този случай може да се извика метода **writer.writerow(user)**. В резултат на това, след като скриптът бъде изпълнен, файлът **users.csv** ще бъде в същата папка, която ще има следното съдържание:

Tom, 28
Alice, 23
Bob, 34
Sam, 31



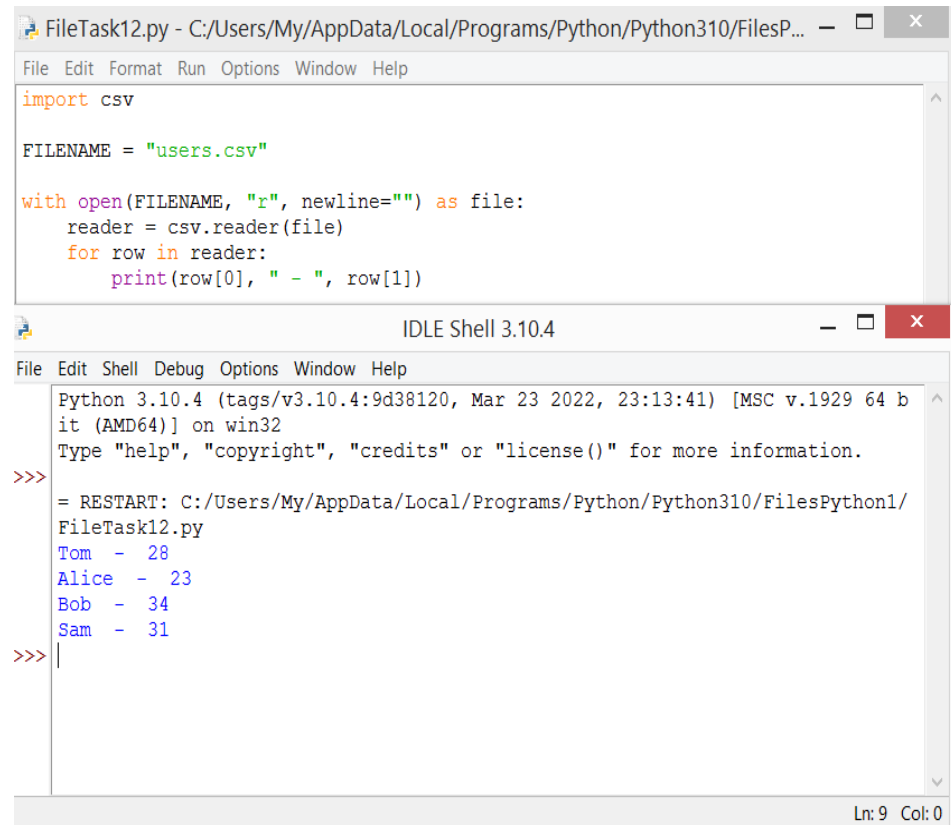
Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

За да се чете от файл, напротив, трябва да се създаде обект за четене:

```
import csv
FILENAME = "users.csv"
with open(FILENAME, "r", newline="") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row[0], " - ", row[1])
```

Когато се получи обект за четене,
може да се премине през всичките му
редове:

```
Tom - 28
Alice - 23
Bob - 34
Sam - 31
```



The screenshot shows a Python IDE window titled 'FileTask12.py'. The code in the editor is as follows:

```
import csv

FILENAME = "users.csv"

with open(FILENAME, "r", newline="") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row[0], " - ", row[1])
```

Below the editor is the 'IDLE Shell 3.10.4' window, which displays the output of the script:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/
FileTask12.py
Tom - 28
Alice - 23
Bob - 34
Sam - 31
>>> |
```

The status bar at the bottom right of the shell window indicates 'Ln: 9 Col: 0'.

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Работа с речници

В примера по-горе всеки запис или ред е отделен списък, като ["Sam", 31]. Но освен това, csv модулът има специални допълнителни функции за работа с речници. По-специално, функцията **csv.DictWriter()** връща обект за запис, който позволява запис във файл. И функцията **csv.DictReader()** връща обект за четене от файла.

Пример:

```
import csv
FILENAME = "users.csv"
users = [
    {"name": "Tom", "age": 28},
    {"name": "Alice", "age": 23},
    {"name": "Bob", "age": 34}
]
```

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

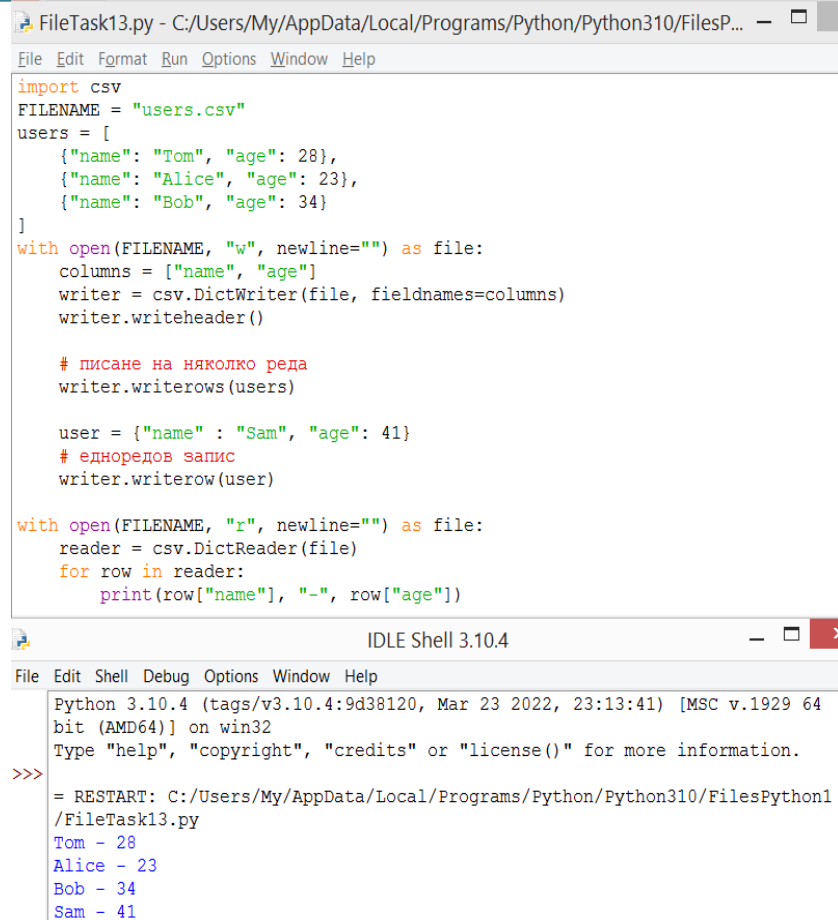
```
with open(FILENAME, "w", newline="") as file:  
    columns = ["name", "age"]  
    writer = csv.DictWriter(file, fieldnames=columns)  
    writer.writeheader()
```

```
# писане на няколко реда  
writer.writerows(users)
```

```
user = {"name": "Sam", "age": 41}  
# едноредов запис  
writer.writerow(user)
```

```
with open(FILENAME, "r", newline="") as file:  
    reader = csv.DictReader(file)  
    for row in reader:  
        print(row["name"], "-", row["age"])
```

Низовете също се записват с помощта на методите **writerow()** и **writerows()**. Но сега всеки ред е отделен речник и освен това заглавията на колоните също се записват с помощта на метода **writeheader()** и набор от колони се предава на метода **csv.DictWriter** като втори параметър. Когато се четат редове, като се използват имена на колони, може да се позовем на отделни стойности в реда: **row["name"]**.



The screenshot shows a Python IDE window titled 'FileTask13.py'. The code defines a CSV file 'users.csv' with columns 'name' and 'age'. It writes a list of users (Tom, Alice, Bob) and then a single user (Sam). Afterward, it reads the file back and prints each row's name and age. The output in the shell shows the data being read: Tom - 28, Alice - 23, Bob - 34, and Sam - 41.

```
FileTask13.py - C:/Users/My/AppData/Local/Programs/Python/Python310/FilesP...  
File Edit Format Run Options Window Help  
import csv  
FILENAME = "users.csv"  
users = [  
    {"name": "Tom", "age": 28},  
    {"name": "Alice", "age": 23},  
    {"name": "Bob", "age": 34}  
]  
with open(FILENAME, "w", newline="") as file:  
    columns = ["name", "age"]  
    writer = csv.DictWriter(file, fieldnames=columns)  
    writer.writeheader()  
  
    # писане на няколко реда  
    writer.writerows(users)  
  
    user = {"name": "Sam", "age": 41}  
    # едноредов запис  
    writer.writerow(user)  
  
with open(FILENAME, "r", newline="") as file:  
    reader = csv.DictReader(file)  
    for row in reader:  
        print(row["name"], "-", row["age"])
```

IDLE Shell 3.10.4
File Edit Shell Debug Options Window Help
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1
/FileTask13.py
Tom - 28
Alice - 23
Bob - 34
Sam - 41

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

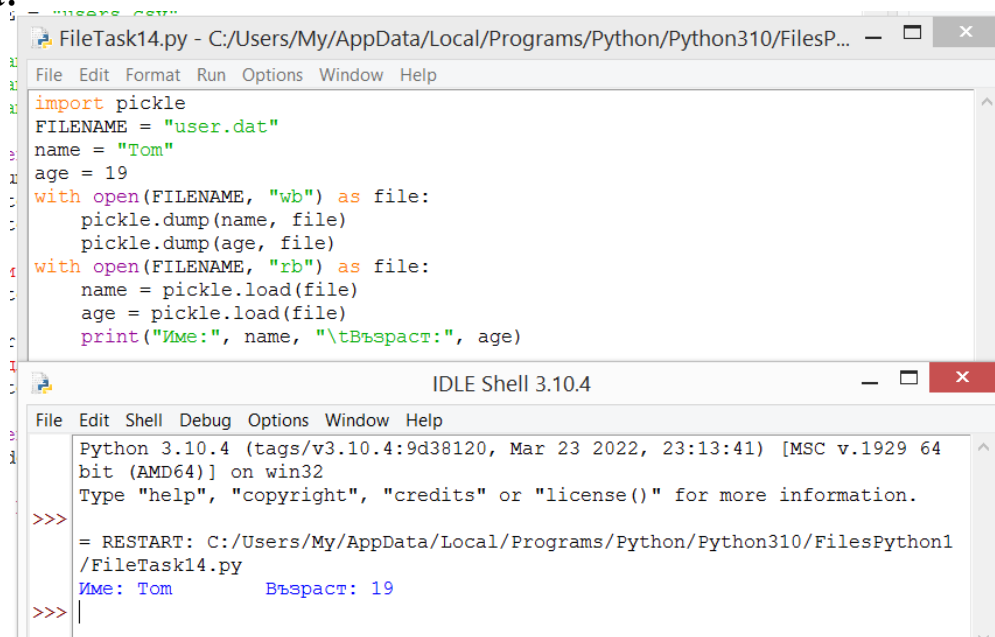
Двоични файлове

Двоичните файлове, за разлика от текстовите, съхраняват информация като набор от байтове. За да работите с тях в Python, е необходим вграденият модул за pickle. Този модул предоставя два метода:

- **dump(obj, file):** записва обекта obj във файла с двоичен файл.
- **load(file):** чете данни от двоичен файл в обект.

Когато се отваря двоичен файл за четене или запис, трябва да се вземе предвид, че трябва да се използва режима "b" в допълнение към режима на запис ("w") или четене ("r"). Ако трябва да се запазят два обекта:

```
import pickle
FILENAME = "user.dat"
name = "Tom"
age = 19
with open(FILENAME, "wb") as file:
    pickle.dump(name, file)
    pickle.dump(age, file)
with open(FILENAME, "rb") as file:
    name = pickle.load(file)
    age = pickle.load(file)
print("Име:", name, "\tВъзраст:", age)
```



The screenshot shows a Python IDE window titled "FileTask14.py" with the following code:

```
import pickle
FILENAME = "user.dat"
name = "Tom"
age = 19
with open(FILENAME, "wb") as file:
    pickle.dump(name, file)
    pickle.dump(age, file)
with open(FILENAME, "rb") as file:
    name = pickle.load(file)
    age = pickle.load(file)
print("Име:", name, "\tВъзраст:", age)
```

Below the code editor is the "IDLE Shell 3.10.4" window, which displays the output of the script:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1
/FileTask14.py
Име: Tom      Възраст: 19
>>>
```


Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Функцията **dump** записва два обекта последователно. Следователно, когато се чете от файл, също може да се четат обектите последователно, като се използва функцията за зареждане.

Конзолен изход на програмата:

Име: Tom Възраст: 28

По същия начин може да се записва и извлича набори от обекти от файл:

```
import pickle
```

```
FILENAME = "users.dat"
```

```
users = [  
    ["Tom", 28, True],  
    ["Alice", 23, False],  
    ["Bob", 34, False]  
]
```

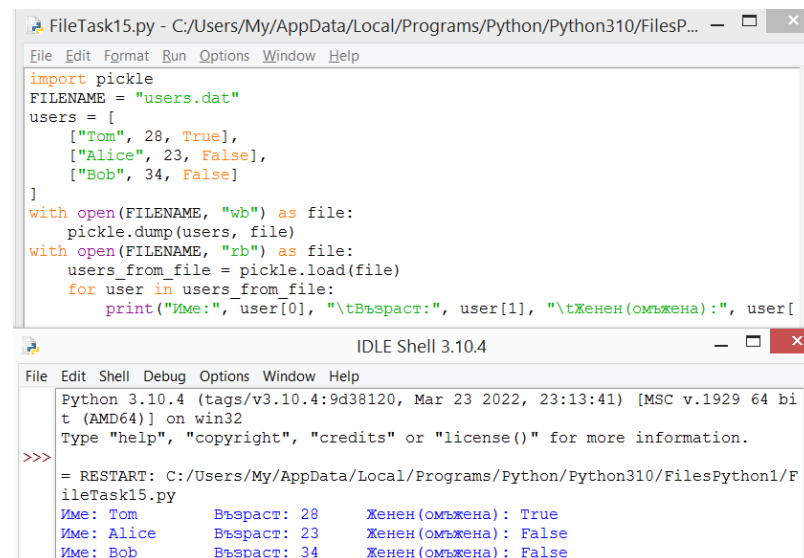
Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

```
with open(FILENAME, "wb") as file:
    pickle.dump(users, file)
with open(FILENAME, "rb") as file:
    users_from_file = pickle.load(file)
    for user in users_from_file:
        print("Име:", user[0], "\tВъзраст:", user[1], "\tЖенен(омъжена):", user[2])
```

В зависимост от това кой обект е изхвърлен, същият обект ще бъде върнат от функцията за зареждане, когато файлът се чете.

Изход на конзолата:

Име: Tom	Възраст: 28	Женен (омъжена): True
Име: Alice	Възраст: 23	Женен (омъжена): False
Име: Bob	Възраст: 34	Женен (омъжена): False



The screenshot shows a Python IDE window titled 'FileTask15.py'. The code in the editor defines a list of users and saves it to a file named 'users.dat' using pickle. It then loads the file back and prints the contents. The console output at the bottom shows the execution of the code, displaying the same user data as the table in the previous block.

```
FileTask15.py - C:/Users/My/AppData/Local/Programs/Python/Python310/FilesP...
File Edit Format Run Options Window Help

import pickle
FILENAME = "users.dat"
users = [
    ["Tom", 28, True],
    ["Alice", 23, False],
    ["Bob", 34, False]
]
with open(FILENAME, "wb") as file:
    pickle.dump(users, file)
with open(FILENAME, "rb") as file:
    users_from_file = pickle.load(file)
    for user in users_from_file:
        print("Име:", user[0], "\tВъзраст:", user[1], "\tЖенен(омъжена):", user[2])

IDLE Shell 3.10.4
File Edit Shell Debug Options Window Help

Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bi
t (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/F
ileTask15.py
Име: Tom      Възраст: 28      Женен(омъжена): True
Име: Alice    Възраст: 23      Женен(омъжена): False
Име: Bob      Възраст: 34      Женен(омъжена): False
```

Рафтов модул (shelve)

Друг модул, който може да се използва за работа с двоични файлове в Python, е `shelve`. Той записва обекти във файл с определен ключ. След това, чрез този ключ, той може да извлече предварително запазения обект от файла. Процесът на работа с данни чрез модула `shelve` е подобен на работата с речници, които също използват ключове за съхранение и извличане на обекти.

Модулът `shelve` използва функцията `open()`, за да отвори файл:
`open(path_to_file[, flag="c", protocol=None[, writeback=False]])`

Където параметърът `flag` може да приема стойности:

- **c:** Файлът се отваря за четене и запис (стойност по подразбиране).
Ако файлът не съществува, тогава той се създава.
- **r:** Файлът се отваря само за четене.
- **w:** Файлът се отваря за запис.
- **n:** Файлът се отваря за запис.

Ако файлът не съществува, той се създава. Ако съществува, тогава се презаписва.

Методът `close()` се извиква за затваряне на файловата връзка:

```
import shelve
d = shelve.open(filename)
d.close()
```

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Може да се отвори файл с изявлението `with`. Нека се запазят и прочетат няколко обекта във файл:

```
import shelve
```

```
FILENAME = "states2"
```

```
with shelve.open(FILENAME) as states:
```

```
    states["London"] = "Great Britain"
```

```
    states["Paris"] = "France"
```

```
    states["Berlin"] = "Germany"
```

```
    states["Madrid"] = "Spain"
```

```
with shelve.open(FILENAME) as states:
```

```
    print(states["London"])
```

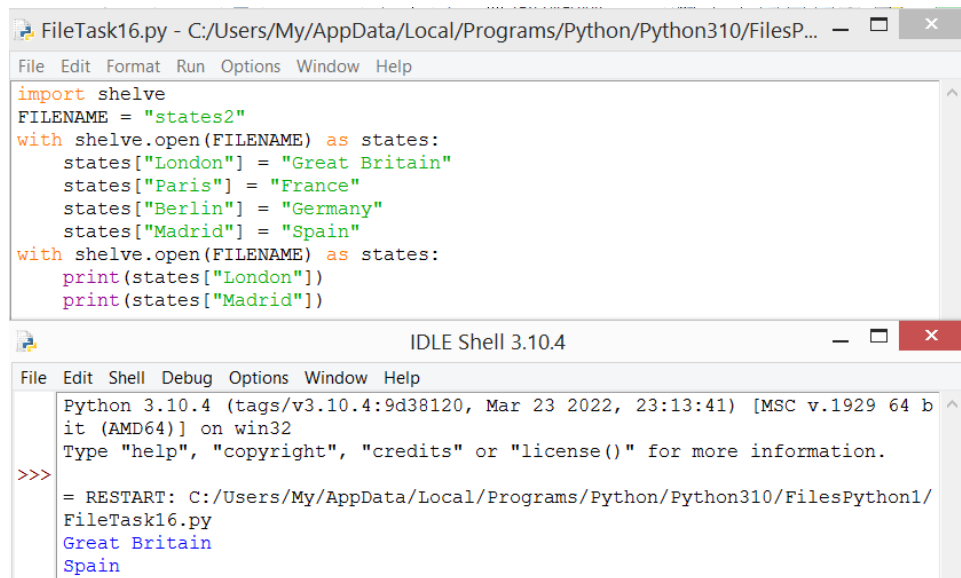
```
    print(states["Madrid"])
```

Записването на данни включва задаване на стойност за конкретен ключ:

```
states["London"] = "Great Britain"
```

Четенето от файл е еквивалентно на получаване на стойност по ключ:

```
print(states["London"])
```



The screenshot shows a Python IDE window titled 'FileTask16.py - C:/Users/My/AppData/Local/Programs/Python/Python310/FilesP...'. The code in the editor is as follows:

```
import shelve
FILENAME = "states2"
with shelve.open(FILENAME) as states:
    states["London"] = "Great Britain"
    states["Paris"] = "France"
    states["Berlin"] = "Germany"
    states["Madrid"] = "Spain"
with shelve.open(FILENAME) as states:
    print(states["London"])
    print(states["Madrid"])
```

Below the editor is the 'IDLE Shell 3.10.4' window. It shows the output of the script:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/
FileTask16.py
Great Britain
Spain
```

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Стойностите на низ се използват като ключове. При четене на данни, ако исканият ключ липсва, се генерира изключение. В този случай, преди да се получи, може да се провери за съществуването на ключа с помощта на оператора `in`:

```
with shelve.open(FILENAME) as states:
```

```
    key = "Brussels"
```

```
    if key in states:
```

```
        print(states[key])
```

Може също да се използва метода `get()`. Първият параметър на метода е ключът, чрез който се получава стойността, а вторият е стойността по подразбиране, която се връща, ако ключът не бъде намерен.

```
with shelve.open(FILENAME) as states:
```

```
    state = states.get("Brussels", "Undefined")
```

```
    print(state)
```

С помощта на цикъл `for` може да се преглеждат всички стойности във файл:

```
with shelve.open(FILENAME) as states:
```

```
    for key in states:
```

```
        print(key, " - ", states[key])
```

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Методът `keys()` връща всички ключове от файла, а методът `values()` връща всички стойности:

with `shelve.open(FILENAME)` as `states`:

for `city` in `states.keys()`:

London Paris Berlin Madrid

print(`city`, end=" ")

print()

for `country` in `states.values()`:

Great Britain France Germany Spain

print(`country`, end=" ")

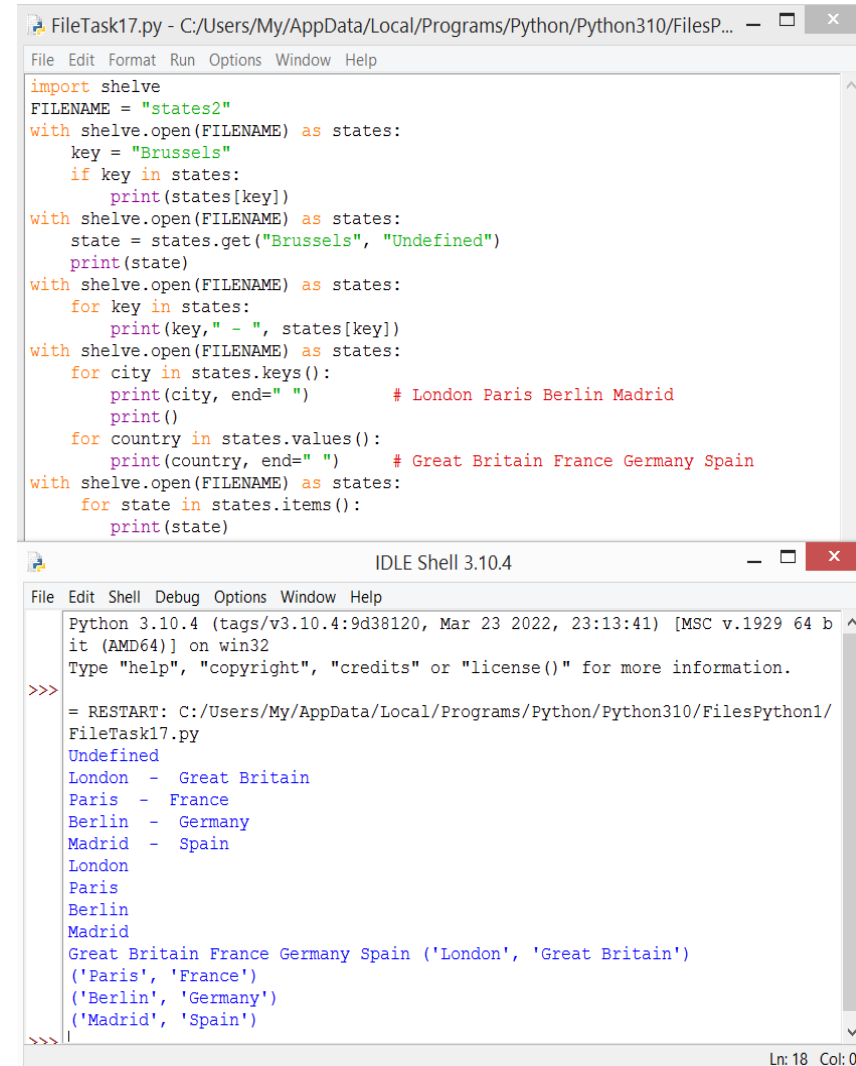
Друг метод `item()` връща набор от кортежи.

Всеки кортеж съдържа ключ и стойност.

with `shelve.open(FILENAME)` as `states`:

for `state` in `states.items()`:

print(`state`)



The screenshot shows a Python IDE with two windows. The top window, titled 'FileTask17.py', contains the following code:

```
import shelve
FILENAME = "states2"
with shelve.open(FILENAME) as states:
    key = "Brussels"
    if key in states:
        print(states[key])
with shelve.open(FILENAME) as states:
    state = states.get("Brussels", "Undefined")
    print(state)
with shelve.open(FILENAME) as states:
    for key in states:
        print(key, " - ", states[key])
with shelve.open(FILENAME) as states:
    for city in states.keys():
        print(city, end=" ")      # London Paris Berlin Madrid
    print()
    for country in states.values():
        print(country, end=" ")  # Great Britain France Germany Spain
with shelve.open(FILENAME) as states:
    for state in states.items():
        print(state)
```

The bottom window, titled 'IDLE Shell 3.10.4', shows the output of the script:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/
FileTask17.py
Undefined
London - Great Britain
Paris - France
Berlin - Germany
Madrid - Spain
London
Paris
Berlin
Madrid
Great Britain France Germany Spain ('London', 'Great Britain')
('Paris', 'France')
('Berlin', 'Germany')
('Madrid', 'Spain')
```

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Актуализация на данните

За да промените данните, достатъчно е да зададете нова стойност на ключа и да добавите данни, за да дефинирате нов ключ:

```
import shelve
```

```
FILENAME = "states2"
```

```
with shelve.open(FILENAME) as states:
```

```
    states["London"] = "Great Britain"
```

```
    states["Paris"] = "France"
```

```
    states["Berlin"] = "Germany"
```

```
    states["Madrid"] = "Spain"
```

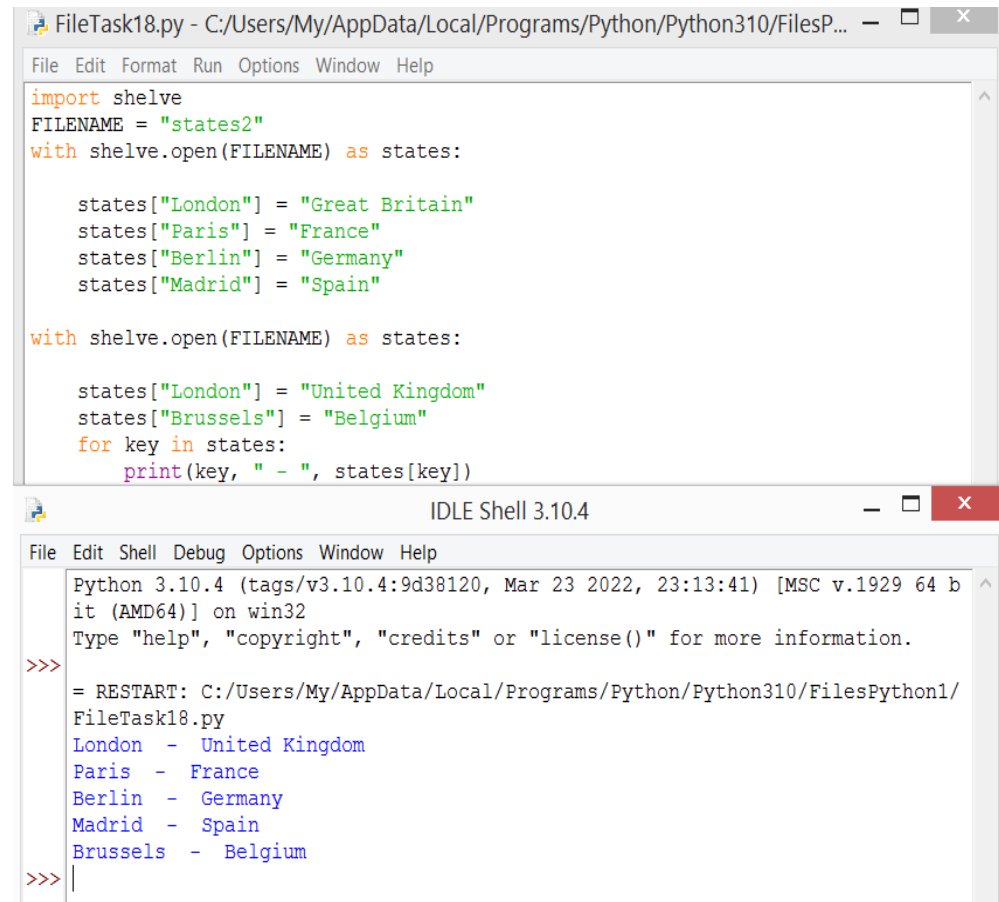
```
with shelve.open(FILENAME) as states:
```

```
    states["London"] = "United Kingdom"
```

```
    states["Brussels"] = "Belgium"
```

```
    for key in states:
```

```
        print(key, " - ", states[key])
```



The screenshot shows two windows from a Python IDE. The top window, titled 'FileTask18.py', contains the following Python code:

```
import shelve
FILENAME = "states2"
with shelve.open(FILENAME) as states:

    states["London"] = "Great Britain"
    states["Paris"] = "France"
    states["Berlin"] = "Germany"
    states["Madrid"] = "Spain"

with shelve.open(FILENAME) as states:

    states["London"] = "United Kingdom"
    states["Brussels"] = "Belgium"
    for key in states:
        print(key, " - ", states[key])
```

The bottom window, titled 'IDLE Shell 3.10.4', shows the output of the script after execution:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 b
it (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/
FileTask18.py
London - United Kingdom
Paris - France
Berlin - Germany
Madrid - Spain
Brussels - Belgium
>>> |
```

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Изтриване на данни

За да премахнете и получите едновременно данни, можете да използвате функцията `pop()`, на която се предава ключът на елемента и стойността по подразбиране, ако ключът не е намерен:

```
import shelve
FILENAME = "states2"
with shelve.open(FILENAME) as states:
    state = states.pop("London", "NotFound")
    print(state)
```

РЕЗУЛТАТ: United Kingdom

Може също да се използва оператора `del`, за изтриване:

```
with shelve.open(FILENAME) as states:
    del states["Madrid"] # изтриване на обект с ключ Madrid
```

РЕЗУЛТАТ: **'Paris', (3072, 21)**
 'Berlin', (3584, 22)
 'Brussels', (4608, 22)

Може да се използва метода `clear()`, за да се премахнат всички елементи:

```
with shelve.open(FILENAME) as states:
    states.clear()
```

РЕЗУЛТАТ: празен файл

OS модул и работа с файловата система

Редица възможности за работа с директории и файлове се предоставят от вградения os модул. Въпреки че съдържа много функции, ще се спрем на няколко основни:

- **mkdir():** създава нова папка;
- **rmdir():** премахва папка;
- **rename():** преименува файл;
- **remove():** премахва файл.

Създаване и изтриване на папка

За да се създаде папка, може да се използва функцията `mkdir()`, на която се предава пътя към създадената папка:

```
import os
# път спрямо текущият скрипт
os.mkdir("hello")
# абсолютен път
os.mkdir("c://somedir")
os.mkdir("c://somedir/hello")
```

Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

За да се изтрие папка, се използва функцията `rmdir()`, на която се предава пътя до папката, която трябва да бъде изтрита:

```
import os
```

```
# път спрямо текущата директория
```

```
os.rmdir("hello")
```

```
# абсолютен път
```

os.rmdir

(**"C://Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/hello"**)

РЕЗУЛТАТ: папка с име **hello** е изтрита

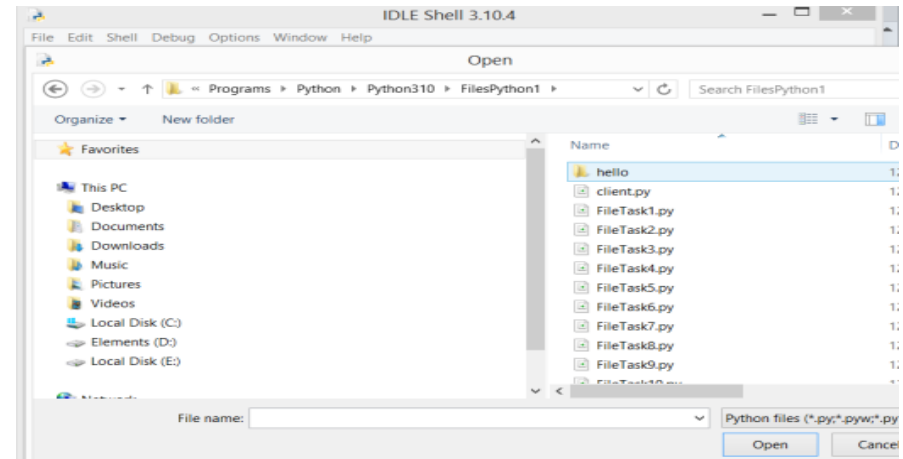
Преименуване на файл

За преименуване се извиква функцията `rename(source, target)`, чийто първи параметър е пътят към изходния файл, а вторият е името на новия файл. Пътищата могат да бъдат абсолютни или относителни. Например, да кажем, че `somefile.txt` се намира в папката `C://SomeDir/`. Преименувайте го във файл `"hello.txt"`:

```
import os
```

```
os.rename("C://Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/hello.tx  
t", "C://Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/somefile.txt")
```

СИНТАКСИС: стар файл, нов файл



Тема 10.2. Работа с файлове. Отваряне и затваряне на файлове. Текстови файлове. CSV файлове Двоични файлове. Рафтов модул

Изтриване на файл

За да се премахне файл, се извиква функцията `remove()`, на която се предава пътя към файла:

```
import os  
os.remove
```

```
("C://Users/My/AppData/Local/Programs/Python/Python310/FilesPython1/somefile.txt")
```

РЕЗУЛТАТ файл с име **somefile.txt** е изтрита

Проверка за съществуващ файл

Ако се опитаме да отворим файл, който не съществува, Python ще хвърли изключение **FileNotFoundError**. За да хванем изключение, може да се използва конструкцията `try...except`. Въпреки това, преди да се отвори файл, може да се провери дали той съществува или не, като се използва метода `os.path.exists(path)`. Пътят, който трябва да бъде проверен, се предава на този метод:

```
filename = input("Въведете път до файла: ")  
if os.path.exists(filename):  
    print("Указаният файл съществува")  
else:  
    print("Файлът не съществува")
```

Използвана литература:

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009